5G finally in India
India is all set to have 70-80 million 5G devices by the end of this year.
https://dgit.in/jul22-61

Samsung's new ISOCELL HP 3 Lens
The new Samsung lens comes up with 200 million pixels and 0.56 micron pixels.
https://dgit.in/jul22-62

Building A Real Time Chat App With Socket.io!

Get cracking with Socket.io

Kshipra Jadav | feedback@digit.in

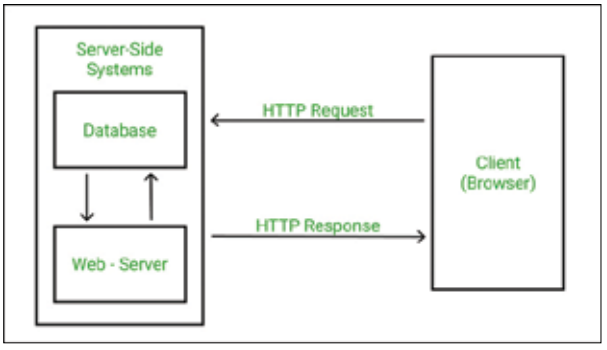
We all know and love chat apps and use them everyday too! WhatsApp, Telegram and even Direct Messages on Instagram – they all act as "Real Time" chat apps. Note that the word "real time" is in focus here. This is so because the delay between point A (when the sender presses the send button) point B (message goes through the server) and point C (the message is received by the receiver) is really low. If you want to get your hands dirty in the chat app world, then this article might just be the one for you. Firstly, we'll give a clear explanation about web sockets and why they're important. Then we'll guide you through making your own real time chat app and then we'll even deploy it on a free service like Heroku.

socket.io

WEB SOCKETS

First, we need to first look at the traditional "request response" model that every client and server follows. The gist is that the client requests the server for some data and the server gives the data back to the client as a response.

So this means that each time you (the client) wants some new data, you have to make and establish a connection then request for the data, wait till the server responds and then successfully close the connection. Now, obviously, this is a very tedious process if you apply this concept in chatting applications. Since each time you press send, the client has to send the data (your text) to the server and the server has to send the data to the receiving side. That is a LOT of connection establishments and connection closings which eventually lead to high resource usage and therefore, will need more time to send each text. If you've still not understood it, let us explain it this way. Say ten guests arrive at your house and are standing on your front gate. Now, if you follow the traditional request response pattern, you will first unlock the door, let ONLY ONE guest in, close the door and lock it. You repeat this ten times until all the guests come inside. This is obviously not the right thing to do since it causes more resource usage and it takes more time for each guest to arrive. Instead, a more efficient way to solve this is to just hold the door open till all the guests come inside and then close it. This way, you'll only unlock, open and lock the door ONCE and all ten guests can enter your house faster.



That is EXACTLY how a web socket works. A web socket establishes a connection with the server and keeps it open. This allows multiple requests and responses to go through and you won't need to establish a new connection for each request and response.

Okay, that's pretty much it for the background story. Let's get our hands dirty with JavaScript and let's start building our Chat App!

BOILERPLATE CODE

The technologies that will be involved in this tutorial are HTML, CSS, JavaScript, NodeJS, ExpressJS, and of course, Socket.io. To build the basic frontend of

our application, we have used HTML, CSS and a bit of JavaScript to get all the buttons and the input fields working on the client side. Also, frontend frameworks and libraries such as React, Angular or Vue have not been used here for the sake of keeping things simple. You can design the frontend according to your likes and dislikes. Since this is not a tutorial for that, we'll consider it as a boilerplate code. Finally, the basic frontend looks like the image we've added above.

For the backend, we've created a basic Express server which serves the